GUÍA TÉCNICA DE DESPLIEGUE NEXT.JS 15.3.9 CON PRISMA 6.19.0 EN CLOUDFLARE PAGES Y WORKERS

Autor: Jhusseth Sanchez
Fecha: Marzo 3, 2026
Versión: 1.0

Resumen

Esta guía técnica proporciona instrucciones detalladas para desplegar una aplicación Next.js 15.3.9 con Prisma 6.19.0 en Cloudflare Pages y Workers utilizando Wrangler CLI. La documentación cubre desde la configuración inicial hasta el despliegue en producción, incluyendo la integración con Cloudflare D1 como base de datos.

1. Introducción

1.1 Objetivo del Documento

Este documento describe el proceso completo de despliegue de una aplicación full-stack Next.js en la infraestructura de Cloudflare, utilizando:

- Next.js 15.3.9 como framework frontend y backend
- Prisma 6.19.0 como ORM para gestión de base de datos
- Cloudflare Pages para hosting estático
- Cloudflare Workers para funciones serverless
- Cloudflare D1 como base de datos SQLite distribuida
- Wrangler CLI como herramienta de despliegue

1.2 Arquitectura de Despliegue

La arquitectura utiliza el adaptador **@opennextjs/cloudflare** que permite ejecutar aplicaciones Next.js con funcionalidades completas (SSR, API Routes, ISR) en Cloudflare Workers[1][7].

Figure 1: Arquitectura de despliegue en Cloudflare

Componentes principales:

- **Cloudflare Pages**: Hosting de assets estáticos (HTML, CSS, JS, imágenes)
- **Cloudflare Workers**: Ejecución de lógica serverless (API Routes, SSR)
- **Cloudflare D1**: Base de datos SQLite distribuida globalmente

- **OpenNext Adapter**: Capa de compatibilidad entre Next.js y Cloudflare

1.3 Requisitos Previos

Componente	Versión Mínima	Versión Recomendada
Node.js	20.0.0	20.x LTS
npm	10.0.0	10.x
Next.js	15.3.9	15.3.9
Prisma	6.19.0	6.19.0
Wrangler	3.99.0	Latest
@opennextjs/cloudflare	-	Latest

Table 1: Requisitos de versiones de software

Cuenta de Cloudflare:

- Cuenta activa en Cloudflare (plan Free o superior)
- API Token con permisos de Workers y Pages
- Dominio configurado (opcional, se puede usar subdominio workers.dev)

2. Configuración Inicial del Proyecto

2.1 Instalación de Dependencias Principales

Primero, instala las dependencias necesarias para el despliegue en Cloudflare[7]:

Instalar adaptador de OpenNext para Cloudflare

```
npm install @opennextjs/cloudflare@latest
```

Instalar Wrangler CLI como dependencia de desarrollo

```
npm install --save-dev wrangler@latest
```

Verificar versiones instaladas

```
npx wrangler --version
```

Nota: Asegúrate de tener Wrangler versión 3.99.0 o superior para compatibilidad con Next.js 15[7].

2.2 Configuración de Next.js

Actualiza tu archivo `next.config.ts` para integrar el adaptador de Cloudflare durante el desarrollo:

// next.config.ts
```
import type { NextConfig } from "next";

const nextConfig: NextConfig = {
// Tus configuraciones existentes
images: {
formats: ['image/avif', 'image/webp'],
},
// Otras configuraciones...
};

export default nextConfig;
```

// Inicializar OpenNext para desarrollo local
```
import { initOpenNextCloudflareForDev } from
"@opennextjs/cloudflare";
initOpenNextCloudflareForDev();
```

Esta configuración permite que el servidor de desarrollo de Next.js se integre óptimamente con el adaptador de Cloudflare y es necesaria para usar bindings durante el desarrollo local[7].

2.3 Configuración de Versión de Node.js

Crea un archivo `.node-version` en la raíz del proyecto para especificar la versión de Node.js[5]:

20

Actualiza tu `package.json` para incluir el motor de Node.js:

```
{
"engines": {
"node": ">=20.0.0"
}
}
```

3. Configuración de Wrangler

3.1 Creación del Archivo wrangler.jsonc

Crea un archivo `wrangler.jsonc` en la raíz de tu proyecto con la siguiente configuración[1][7]:

```
{
"$schema": "node_modules/wrangler/config-schema.json",
"name": "mi-app-nextjs",
"main": ".open-next/worker.js",
"compatibility_date": "2026-03-01",
"compatibility_flags": [
"nodejs_compat"
],
"observability": {
"enabled": true
}
}
```

Explicación de parámetros:

Parámetro	Descripción
name	Nombre de tu Worker en Cloudflare
main	Punto de entrada generado por OpenNext
compatibility_date	Fecha de compatibilidad de APIs de Workers
nodejs_compat	Flag esencial para APIs de Node.js en Workers[8]
observability	Habilita logs y monitoreo en producción

3.2 Variables de Entorno

Para gestionar variables de entorno en diferentes ambientes, actualiza wrangler.jsonc:

```
{
"$schema": "node_modules/wrangler/config-schema.json",
"name": "mi-app-nextjs",
"main": ".open-next/worker.js",
"compatibility_date": "2026-03-01",
"compatibility_flags": ["nodejs_compat"],

"vars": {
"ENVIRONMENT": "production",
"API_URL": "https://api.midominio.com"
},

"env": {
"staging": {
"name": "mi-app-nextjs-staging",
"vars": {
"ENVIRONMENT": "staging",
"API_URL": "https://api-staging.midominio.com"
}
}
}
}
```

4. Configuración de Prisma con Cloudflare D1

4.1 Instalación de Dependencias de Prisma

Instala Prisma y el adaptador para Cloudflare D1[6][9]:

Instalar Prisma Client y adaptador D1

```
npm install @prisma/client @prisma/adapter-d1
```

Instalar Prisma CLI como dependencia de desarrollo

```
npm install --save-dev prisma
```

4.2 Configuración del Schema de Prisma

Crea o actualiza prisma/schema.prisma con la configuración para D1[9]:

```
generator client {
provider = "prisma-client-js"
previewFeatures = ["driverAdapters"]
}

datasource db {
provider = "sqlite"
url = env("DATABASE_URL")
}

// Modelos de ejemplo
model User {
id Int @id @default(autoincrement())
email String @unique
name String?
createdAt DateTime @default(now())
posts Post[]
}

model Post {
id Int @id @default(autoincrement())
title String
content String?
published Boolean @default(false)
authorId Int
author User @relation(fields: [authorId], references: [id])
createdAt DateTime @default(now())
}
```

4.3 Creación de Base de Datos D1

Autentica con Cloudflare y crea una base de datos D1[6]:

Autenticar con Cloudflare

```
npx wrangler login
```

Crear base de datos D1

```
npx wrangler d1 create mi-app-database
```

El comando devolverá información similar a:

✅ Successfully created DB 'mi-app-database' in region WEUR
Created your database using D1's new storage backend.

```
[[d1_databases]]
binding = "DB"
database_name = "mi-app-database"
database_id = "xxxx-xxxx-xxxx-xxxx-xxxx"
```

4.4 Binding de D1 en Wrangler

Agrega la configuración de D1 a tu wrangler.jsonc[6]:

```
{
"$schema": "node_modules/wrangler/config-schema.json",
"name": "mi-app-nextjs",
"main": ".open-next/worker.js",
"compatibility_date": "2026-03-01",
"compatibility_flags": ["nodejs_compat"],

"d1_databases": [
{
"binding": "DB",
"database_name": "mi-app-database",
"database_id": "xxxx-xxxx-xxxx-xxxx-xxxx"
}
]
}
```

4.5 Generación y Aplicación de Migraciones

Genera las migraciones de Prisma y aplícalas a D1[6]:

Generar migraciones de Prisma

```
npx prisma migrate dev --name init
```

Aplicar migraciones a base de datos local D1

```
npx wrangler d1 execute mi-app-database --local --
file=./prisma/migrations/XXXXXX_init/migration.sql
```

Aplicar migraciones a base de datos remota D1

```
npx wrangler d1 execute mi-app-database --remote --
file=./prisma/migrations/XXXXXX_init/migration.sql
```

Nota: Reemplaza `XXXXXX_init` con el nombre real de tu carpeta de migración.

4.6 Configuración del Cliente Prisma para D1

Crea un archivo `lib/prisma.ts` para inicializar el cliente Prisma con D1[9]:

```
// lib/prisma.ts
import { PrismaClient } from '@prisma/client';
import { PrismaD1 } from '@prisma/adapter-d1';
```

```
export function getPrismaClient(db: D1Database) {
const adapter = new PrismaD1(db);
const prisma = new PrismaClient({ adapter });
return prisma;
}

Uso en API Routes de Next.js:

// app/api/users/route.ts
import { getPrismaClient } from '@/lib/prisma';
import { NextRequest } from 'next/server';

export const runtime = 'edge';

export async function GET(request: NextRequest) {
// Obtener binding D1 desde el entorno de Cloudflare
const db = (request as any).env.DB;
const prisma = getPrismaClient(db);

const users = await prisma.user.findMany();
return Response.json(users);
}
```

5. Configuración de OpenNext

5.1 Creación del Archivo open-next.config.ts

Crea un archivo open-next.config.ts en la raíz del proyecto[7]:

```
// open-next.config.ts
import type { OpenNextConfig } from '@opennextjs/cloudflare';

const config: OpenNextConfig = {
default: {
override: {
wrapper: 'cloudflare-node',
converter: 'edge',
incrementalCache: 'dummy',
},
},
};

export default config;
```

5.2 Configuración Avanzada con KV y R2

Si necesitas usar Cloudflare KV para caché incremental o R2 para almacenamiento de objetos:

```
// open-next.config.ts
import type { OpenNextConfig } from '@opennextjs/cloudflare';

const config: OpenNextConfig = {
default: {
```

```
override: {
wrapper: 'cloudflare-node',
converter: 'edge',
incrementalCache: 'kv', // Usar KV para ISR
},
},
};

export default config;

Agrega los bindings correspondientes en wrangler.jsonc:

{
"kv_namespaces": [
{
"binding": "CACHE",
"id": "tu-kv-namespace-id"
}
],
"r2_buckets": [
{
"binding": "ASSETS",
"bucket_name": "mi-app-assets"
}
]
}
```

6. Scripts de Build y Deploy

6.1 Configuración de package.json

Actualiza los scripts en tu `package.json` para incluir comandos de build y deploy[7]:

```
{
"scripts": {
"dev": "next dev",
"build": "next build",
"start": "next start",
"lint": "next lint",

"pages:build": "npx @cloudflare/next-on-pages",
"pages:deploy": "npm run pages:build && wrangler pages deploy
.vercel/output/static",
"pages:dev": "npx @cloudflare/next-on-pages --watch",

"cf:build": "next build",
"cf:deploy": "next build && wrangler deploy",
"cf:dev": "wrangler dev",

"db:generate": "prisma generate",
"db:migrate:dev": "prisma migrate dev",
"db:migrate:deploy": "wrangler d1 execute mi-app-database --remote -
-file=./prisma/migrations/LATEST/migration.sql",
```

```
"db:studio": "prisma studio"


}
}
```

Explicación de scripts:

Script	Función
cf:build	Construye la aplicación Next.js
cf:deploy	Construye y despliega a Cloudflare Workers
cf:dev	Inicia servidor de desarrollo local con Wrangler
db:migrate:deploy	Aplica migraciones a D1 en producción

7. Proceso de Despliegue

7.1 Autenticación con Cloudflare

Antes del primer despliegue, autentica tu CLI con Cloudflare[6]:

```
npx wrangler login
```

Este comando abrirá tu navegador para autorizar el acceso de Wrangler a tu cuenta de Cloudflare.

7.2 Despliegue Inicial

Ejecuta el comando de despliegue:

```
npx wrangler deploy
```

O usando el script configurado:

```
npm run cf:deploy
```

Proceso de despliegue:

1. Next.js construye la aplicación (build estático y serverless)
2. OpenNext adapta el output para Cloudflare Workers
3. Wrangler empaqueta y sube los assets a Cloudflare
4. Se crean los Workers necesarios para SSR y API Routes
5. Se aplican los bindings configurados (D1, KV, R2, etc.)
6. Se genera la URL de producción

7.3 Verificación del Despliegue

Después del despliegue exitoso, verás un output similar a:

✨ Success! Deployed to https://mi-app-nextjs.username.workers.dev

Verifica tu aplicación accediendo a la URL proporcionada y prueba:

- Páginas estáticas
- Server-Side Rendering (SSR)
- API Routes
- Conexión a base de datos D1
- Autenticación (si aplica)

7.4 Despliegue de Migraciones de Base de Datos

Antes de desplegar cambios que requieran actualizaciones de schema:

1. Generar nueva migración

```
npx prisma migrate dev --name descripcion_cambio
```

2. Aplicar a base de datos local para testing

```
npx wrangler d1 execute mi-app-database --local --
file=./prisma/migrations/XXXXXX_descripcion_cambio/migration.sql
```

3. Aplicar a producción

```
npx wrangler d1 execute mi-app-database --remote --
file=./prisma/migrations/XXXXXX_descripcion_cambio/migration.sql
```

4. Desplegar aplicación actualizada

```
npm run cf:deploy
```

8. Configuración de Dominios Personalizados

8.1 Configuración en Dashboard de Cloudflare

Para usar un dominio personalizado en lugar de workers.dev:

1. Accede al Dashboard de Cloudflare
2. Ve a **Workers & Pages**
3. Selecciona tu aplicación desplegada
4. Haz clic en **Settings** > **Domains & Routes**
5. Haz clic en **Add Custom Domain**
6. Ingresa tu dominio (ej: app.midominio.com)
7. Cloudflare configurará automáticamente los registros DNS

8.2 Configuración mediante CLI

También puedes configurar rutas personalizadas usando Wrangler:

npx wrangler routes add app.midominio.com/* mi-app-nextjs

9. Entornos Múltiples (Staging y Producción)

9.1 Configuración de Entornos

Actualiza wrangler.jsonc para soportar múltiples entornos[13]:

```
{
"$schema": "node_modules/wrangler/config-schema.json",
"name": "mi-app-nextjs-prod",
"main": ".open-next/worker.js",
"compatibility_date": "2026-03-01",
"compatibility_flags": ["nodejs_compat"],

"vars": {
"ENVIRONMENT": "production"
},

"d1_databases": [
{
"binding": "DB",
"database_name": "mi-app-database-prod",
"database_id": "prod-database-id"
}
],

"env": {
"staging": {
"name": "mi-app-nextjs-staging",
"vars": {
"ENVIRONMENT": "staging"
},
"d1_databases": [
{
"binding": "DB",
"database_name": "mi-app-database-staging",
"database_id": "staging-database-id"
}
]
},
"development": {
"name": "mi-app-nextjs-dev",
"vars": {
"ENVIRONMENT": "development"
},
"d1_databases": [
{
```

```
"binding": "DB",
"database_name": "mi-app-database-dev",
"database_id": "dev-database-id"
}
]
}
}
}
```

9.2 Despliegue a Entornos Específicos

Despliega a diferentes entornos usando el flag `--env`:

Desplegar a producción (default)

```
npx wrangler deploy
```

Desplegar a staging

```
npx wrangler deploy --env staging
```

Desplegar a desarrollo

```
npx wrangler deploy --env development
```

10. Desarrollo Local

10.1 Servidor de Desarrollo con Wrangler

Para desarrollar localmente con acceso a bindings de Cloudflare:

```
npx wrangler dev
```

Este comando:

- Inicia el servidor de desarrollo de Next.js
- Emula el entorno de Cloudflare Workers localmente
- Proporciona acceso a D1, KV, R2 y otros bindings
- Permite hot-reload durante el desarrollo

10.2 Testing con Base de Datos Local D1

Wrangler puede crear una instancia local de D1 para desarrollo:

Ejecutar en modo desarrollo con D1 local

npx wrangler dev --local

Abrir consola de D1 local

```
npx wrangler d1 execute mi-app-database --local --command="SELECT *
FROM User"
```

11. Monitoreo y Logs

11.1 Visualización de Logs en Tiempo Real

Monitorea los logs de tu Worker en tiempo real:

npx wrangler tail

Para un entorno específico:

npx wrangler tail --env staging

11.2 Dashboard de Cloudflare

Accede a métricas y análisis en el Dashboard:

1. Ve a **Workers & Pages**
2. Selecciona tu aplicación
3. Accede a **Metrics** para ver:
    - Requests por segundo
    - Errores y excepciones
    - CPU time y duración de ejecución
    - Datos transferidos

12. Optimizaciones y Mejores Prácticas

12.1 Optimización de Build

Configura Next.js para optimizar el tamaño del bundle:

```
// next.config.ts
const nextConfig: NextConfig = {
output: 'export', // Solo si usas exportación estática

compiler: {
removeConsole: process.env.NODE_ENV === 'production',
},

experimental: {
optimizePackageImports: ['@prisma/client'],
},
};
```

12.2 Caché y Performance

Implementa estrategias de caché en tus API Routes:

```
// app/api/data/route.ts
export const runtime = 'edge';

export async function GET() {
const data = await fetchData();

return Response.json(data, {
headers: {
'Cache-Control': 'public, s-maxage=3600, stale-while-
revalidate=86400',
},
});
}
```

12.3 Límites de Cloudflare Workers

Ten en cuenta los límites del plan Free[1]:

Recurso	Límite (Plan Free)
CPU Time	10ms por request
Memory	128MB
Script Size	1MB comprimido
Requests	100,000 por día
D1 Rows Read	5M por día
D1 Rows Written	100K por día

13. Troubleshooting

13.1 Error: "Module not found"

Problema: Módulos Node.js no disponibles en Workers.

Solución: Verifica que tienes el flag `nodejs_compat` en `wrangler.jsonc`[8]:

```
{
"compatibility_flags": ["nodejs_compat"]
}
```

13.2 Error de Conexión a D1

Problema: No se puede acceder a la base de datos D1.

Soluciones:

1. Verifica que el binding esté configurado correctamente en `wrangler.jsonc`
2. Asegúrate de pasar el binding correcto al cliente Prisma
3. Verifica que las migraciones estén aplicadas:

```
npx wrangler d1 execute mi-app-database --remote --command="SELECT
name FROM sqlite_master WHERE type='table'"
```

13.3 Build Failures

Problema: Falla el build durante el despliegue.

Soluciones:

1. Verifica las versiones de dependencias
2. Ejecuta build local para identificar errores:

   ```
   npm run build
   ```

3. Revisa los logs de error y verifica compatibilidad con Edge Runtime
4. Asegúrate de no usar APIs incompatibles (fs, path, child_process)[8]

13.4 Errores de Prisma

Problema: Prisma no genera el cliente correctamente.

Soluciones:

Regenerar cliente Prisma

```
npx prisma generate
```

Limpiar y reinstalar

```
rm -rf node_modules .next
npm install
npx prisma generate
```

14. CI/CD con GitHub Actions

14.1 Configuración de GitHub Actions

Crea un archivo .github/workflows/deploy.yml:

```
name: Deploy to Cloudflare

on:
push:
branches:
```

```
- main
- staging

jobs:
deploy:
runs-on: ubuntu-latest
name: Deploy
steps:
- uses: actions/checkout@v4

  - name: Setup Node.js
    uses: actions/setup-node@v4
    with:
      node-version: '20'
      cache: 'npm'

  - name: Install dependencies
    run: npm ci

  - name: Generate Prisma Client
    run: npx prisma generate

  - name: Build application
    run: npm run build

  - name: Deploy to Cloudflare Workers
    uses: cloudflare/wrangler-action@v3
    with:
      apiToken: ${{ secrets.CLOUDFLARE_API_TOKEN }}
      accountId: ${{ secrets.CLOUDFLARE_ACCOUNT_ID }}
      command: deploy --env ${{ github.ref == 'refs/heads/main'
&& 'production' || 'staging' }}
```

14.2 Configuración de Secrets

En tu repositorio de GitHub, agrega los siguientes secrets:

1. Ve a **Settings** > **Secrets and variables** > **Actions**
2. Agrega:
   - `CLOUDFLARE\_API\_TOKEN: Token de API de Cloudflare`
   - `CLOUDFLARE\_ACCOUNT\_ID: ID de tu cuenta de Cloudflare`

15. Seguridad

15.1 Variables de Entorno Sensibles

Nunca commitees información sensible en `wrangler.jsonc`. Usa secrets:

Agregar secret

```
npx wrangler secret put DATABASE_URL
```

Listar secrets

```
npx wrangler secret list
```

Eliminar secret

```
npx wrangler secret delete DATABASE_URL
```

15.2 CORS Configuration

Configura CORS en tus API Routes:

```
// middleware.ts
import { NextResponse } from 'next/server';
import type { NextRequest } from 'next/server';

export function middleware(request: NextRequest) {
const response = NextResponse.next();

response.headers.set('Access-Control-Allow-Origin',
'https://tudominio.com');
response.headers.set('Access-Control-Allow-Methods', 'GET, POST,
PUT, DELETE, OPTIONS');
response.headers.set('Access-Control-Allow-Headers', 'Content-Type,
Authorization');

return response;
}

export const config = {
matcher: '/api/:path*',
};
```

15.3 Rate Limiting

Implementa rate limiting usando Cloudflare Workers:

```
// lib/rate-limit.ts
export async function rateLimit(request: Request, kv: KVNamespace) {
const ip = request.headers.get('CF-Connecting-IP') || 'unknown';
const key = rate-limit:${ip};

const count = await kv.get(key);
const currentCount = count ? parseInt(count) : 0;

if (currentCount > 100) {
return new Response('Too Many Requests', { status: 429 });
}

await kv.put(key, String(currentCount + 1), { expirationTtl: 3600
});
```

```
return null;
}
```

16. Conclusiones

16.1 Resumen del Proceso

El despliegue de Next.js 15.3.9 con Prisma 6.19.0 en Cloudflare requiere:

1. Instalación de @opennextjs/cloudflare y Wrangler CLI
2. Configuración de wrangler.jsonc con flags de compatibilidad
3. Setup de Prisma con adaptador D1
4. Creación y binding de base de datos D1
5. Aplicación de migraciones a D1
6. Despliegue usando `npx wrangler deploy`

16.2 Ventajas de Cloudflare

- Distribución global con baja latencia
- Escalabilidad automática
- Costos predecibles (plan Free generoso)
- Integración nativa con servicios Cloudflare (D1, KV, R2, etc.)
- Soporte completo para Next.js SSR y API Routes

16.3 Siguientes Pasos

1. Configurar dominio personalizado
2. Implementar CI/CD con GitHub Actions
3. Configurar múltiples entornos (staging, production)
4. Implementar monitoreo y alertas
5. Optimizar performance con caché avanzado
6. Configurar backups de base de datos D1

Referencias

[1] Cloudflare Workers. (2026). Next.js · Cloudflare Workers docs.
https://developers.cloudflare.com/workers/framework-guides/web-apps/nextjs/

[2] Cloudflare. (2026). Next.js · Cloudflare Pages docs.
https://developers.cloudflare.com/pages/framework-guides/nextjs/

[3] Penchef. (2026). Deploying Next.js to Cloudflare Pages: A Walkthrough. https://www.penchef.com/devops-and-cloud-infrastructure/deploying-nextjs-cloudflare-pages-walkthrough

[4] Ram Patra. (2024). How to run a website built with Next.js 15 on Cloudflare Pages. https://blog.rampatra.com/how-to-run-a-website-built-with-next-js-15-on-cloudflare-pages

[5] Vercel. (2024). Next.js 15. https://nextjs.org/blog/next-15

[6] Prisma. (2026). How to use Prisma ORM with Cloudflare D1. https://www.prisma.io/docs/guides/deployment/cloudflare-d1

[7] OpenNext. (2026). Get Started - OpenNext. https://opennext.js.org/cloudflare/get-started

[8] Penchef. (2026). Deploying Next.js to Cloudflare Pages: Technical Walkthrough. https://www.penchef.com/devops-and-cloud-infrastructure/deploying-nextjs-cloudflare-pages-walkthrough

[9] Zenn. (2024). Cloudflare Workers から Prisma で D1 を操作する. https://zenn.dev/ss49919201/articles/72f37427446d12

[10] Dev.to. (2025). Deploying Next.js App to Cloudflare Workers with OpenNext. https://dev.to/prajwolshrestha/deploying-nextjs-app-to-cloudflare-workers-with-opennext-hi0

[11] Devin Vinson. (2025). Deploying Next.js on Cloudflare: My Troubleshooting Guide. https://devinvinson.com/2025/11/deploying-fullstack-next-js-on-cloudflare-my-troubleshooting-guide/

[12] Prisma. (2026). Cloudflare D1 Integration Guide. https://www.prisma.io/docs/guides/cloudflare-d1

[13] Reddit. (2025). How to deploy staging and production branches on Cloudflare Workers with Next.js. https://www.reddit.com/r/CloudFlare/comments/1myzw4i/

Apéndices

Apéndice A: Checklist de Despliegue

- ☐ Node.js 20.x instalado
- ☐ Cuenta de Cloudflare activa
- ☐ Wrangler CLI instalado (v3.99.0+)
- ☐ @opennextjs/cloudflare instalado
- ☐ wrangler.jsonc configurado
- ☐ open-next.config.ts creado
- ☐ Base de datos D1 creada
- ☐ Bindings configurados en wrangler.jsonc
- ☐ Migraciones de Prisma aplicadas
- ☐ Build local exitoso

- ☐ Testing local con wrangler dev
- ☐ Autenticación con Cloudflare completada
- ☐ Despliegue ejecutado
- ☐ Verificación en producción
- ☐ Dominio personalizado configurado (opcional)
- ☐ CI/CD configurado (opcional)

Apéndice B: Comandos Útiles de Referencia Rápida

Instalación

```
npm install @opennextjs/cloudflare@latest
npm install --save-dev wrangler@latest
```

Autenticación

```
npx wrangler login
```

Base de datos D1

```
npx wrangler d1 create mi-app-database
npx wrangler d1 execute DB --local --file=migration.sql
npx wrangler d1 execute DB --remote --file=migration.sql
```

Prisma

```
npx prisma generate
npx prisma migrate dev --name init
```

Desarrollo

```
npx wrangler dev
npx wrangler dev --local
```

Despliegue

```
npx wrangler deploy
npx wrangler deploy --env staging
npx wrangler deploy --env production
```

Logs y monitoreo

```
npx wrangler tail
npx wrangler tail --env staging
```

Secrets

```
npx wrangler secret put SECRET_NAME
npx wrangler secret list
npx wrangler secret delete SECRET_NAME
```

Dominios

```
npx wrangler routes list
npx wrangler routes add example.com/* worker-name
```

Apéndice C: Estructura de Archivos Recomendada

```
mi-app-nextjs/
├── .github/
│   └── workflows/
│   └── deploy.yml
├── app/
│   ├── api/
│   │   └── [...routes]/
│   ├── (pages)/
│   └── layout.tsx
├── lib/
│   ├── prisma.ts
│   └── utils.ts
├── prisma/
│   ├── migrations/
│   └── schema.prisma
├── public/
├── .node-version
├── next.config.ts
├── open-next.config.ts
├── wrangler.jsonc
├── package.json
├── tsconfig.json
└── README.md
```